

title:	Own Your Team Secrets Like You Own Your Code
date:	2026-07-06
status:	draft
platform:	linkedin

Most teams rent secret sharing from SaaS: seats, dashboards, org roles, hosted vaults, and another monthly bill.

I wanted the opposite.

Own the encrypted repo. Own the keys. Add and remove teammates with the SSH keys your team already uses. Let humans and coding agents use the same CLI to encrypt and decrypt secrets. Pay nothing per seat.

That's the offer behind `sshare` : **own your team secrets the way you own your code.**

Technically, it is a secret-sharing CLI with no auth server, no accounts, no roles table, and no permissions middleware.

The access check is a failed decryption.

`sshare` shares passwords, API tokens, and `.env` files by encrypting them to the team's SSH public keys. The

encrypted blobs live in a git repository. If you hold a matching SSH private key, `sshare get` works. If you don't, it doesn't.

That sounds almost too simple. The interesting part was keeping it simple after the first obvious security hole appeared.

The constraint: git repo + SSH keys, nothing else

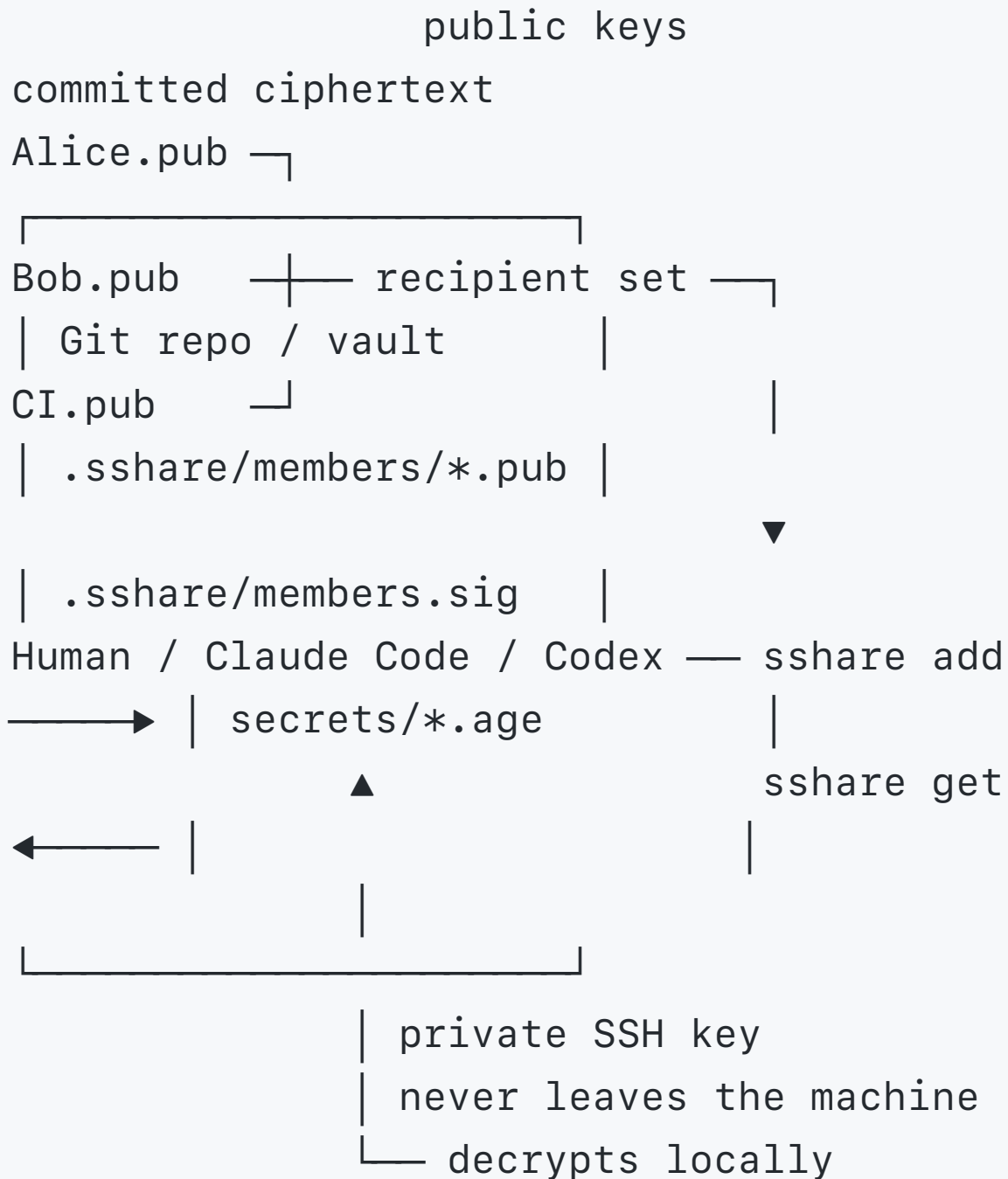
I wanted a tool that fits into the workflow teams already use:

```
sshare member add alice --key
~/.ssh/id_ed25519.pub
printf 'secret-value' | sshare add db-prod
sshare get db-prod
sshare git push
```

No hosted service. No new identity provider. No per-seat billing. No external `age` or `gpg` binary. Just a single Rust CLI, a vault layout in a repo, and the SSH keys developers already understand.

Because it is a normal CLI, the same interface works for people and agents: Claude Code, Codex, a shell script, or CI can all use the same two primitives — `sshare add` to encrypt and `sshare get` to decrypt — as long as the

machine has the right private key.



Remove Bob:

members - Bob.pub → rekey → new *.age files
encrypted only to remaining keys

The current implementation is deliberately small: 8 Rust source files, about 2,234 lines of source, 30 tests, and a release binary built around two isolated crypto modules:

- `crypto.rs` is the only place that knows about `age`
- `sign.rs` is the only place that knows about SSH signatures
- `vault.rs` owns filesystem layout and name validation
- `main.rs` handles CLI I/O and command flow

That boundary matters because in a secrets tool, “where crypto lives” is an architectural decision, not just code organization.

The core model: access control is the recipient set

The encryption path is intentionally boring:

```
pub(crate) fn encrypt(
    plaintext: &[u8],
    recipients: &[age::ssh::Recipient],
) -> Result<Vec<u8>> {
    if recipients.is_empty() {
        bail!("cannot encrypt: no
recipients");
```

```

    }

    let encryptor =
age::Encryptor::with_recipients(
    recipients.iter().map(|r| r as &dyn
age::Recipient),
    )?;

    // write plaintext into age stream...
}

```

Every stored secret is encrypted to every member's SSH public key. Decryption only uses the caller's private key. There is no extra "can this user read this?" check because any check outside the ciphertext would be weaker than the ciphertext.

The rule became: if a feature looks like authorization, express it as recipients + re-encryption.

That rule keeps the product honest. Removing a member is not "set active = false." It means removing their public key from the member set and running `rekey`, which decrypts each secret with your key and re-encrypts it to the current members.

It also keeps the limitations honest: revocation stops future

access, but it cannot make someone forget a secret they already decrypted. The tool prints that warning because pretending otherwise would be security theater.

The first real flaw: who protects the member list?

The naive design had a problem.

If secrets are encrypted to every public key in

`.sshare/members/`, then anyone with git write access could add a key:

```
cp ~/.ssh/id_ed25519.pub  
.sshare/members/mallory.pub  
git commit -am "update members"  
git push
```

They still couldn't decrypt old secrets. But the next `sshare add` or `sshare rekey` would encrypt future secrets to Mallory too.

So the model had two different bars:

- **Can decrypt:** requires a private key
- **Can grant future decryption:** requires only a commit

That gap was unacceptable.

The fix was not to add a server. The fix was to authenticate the recipient set.

Tamper-evident membership with SSHSIG + TOFU

`sshare` now signs the canonical member list with a maintainer's SSH key. The signature is committed into the repo as `.sshare/members.sig`.

Each machine pins the maintainer fingerprint on first use — TOFU, similar to SSH `known_hosts` — but the pin lives outside the repo in the user's config directory. That separation is the protection: a repo committer can change files in the repo, but not your local trust pin.

Before encrypting, `add` and `rekey` verify the member list:

```
fn verify_members_trusted(vault: &Vault) ->
Result<()> {
    let sig =
vault.read_members_sig()?.ok_or_else(|| {
        anyhow!("this vault's member list is
not signed")
    })?;
```

```

    let signer =
sign::verify(&vault.canonical_members()?,
&sig)
        .context("the member list signature is
invalid"?);

    match
TrustStore::load()?.pinned(&vault.vault_id()?)
{
    Some(pinned) if pinned == signer =>
Ok(()),
    Some(pinned) => bail!("signed by
{signer}, pinned authority is {pinned}"),
    None => bail!("signing authority
({signer}) is not yet trusted"),
    }
}

```

The important point: this is still not a roles system.

It does not say "Alice is allowed to add Bob." It says "this recipient set is the one signed by the authority I trust." Then encryption uses that recipient set. Access control remains cryptographic.

TOFU has a weak first moment: if your very first clone is

already tampered with, you can pin the wrong authority. The mitigation is the same as SSH host keys: verify the fingerprint out-of-band. Not perfect, but explicit.

Git is transport, not a backend

The repo is just the synchronization layer. GitHub, GitLab, or any other host only sees ciphertext, member public keys, signatures, and secret names.

`sshare` auto-commits local mutations so users don't forget to record changes:

- `sshare: add secret db-prod`
- `sshare: remove member bob`
- `sshare: rekey 12 secret(s) for 3 member(s)`

But it never auto-pushes or auto-pulls. Network happens only when the user explicitly runs `sshare git push` or `sshare git pull`.

That line matters for scripts and agents. Reads don't commit. Reads don't hit the network. Mutations are local unless you ask git to sync.

This is what makes the tool agent-friendly: an agent does not need a browser session, a SaaS token, or a new integration. It can operate the vault through ordinary commands, while the

private key still stays on the machine where decryption happens.

The small details are where trust is lost

A few design decisions looked minor but ended up defining the product:

Names are visible, values are not. Secret names leak to anyone with repo access, so the docs tell users not to put sensitive data in names.

Descriptions are encrypted too. A note like "Stripe live key for PII export job" is often more sensitive than the name itself. In v0.6.0, descriptions became encrypted sidecar blobs, rekeyed with the secret.

get writes raw bytes to stdout. No JSON envelope, no metadata wrapper, no formatting. This makes it pipe cleanly into `.env` files and scripts.

Path traversal is rejected before paths are built. Secret names can be nested (`prod/api-token`), but every component must be a safe name:

```
fn validate_component(component: &str) ->
Result<()> {
    if component.is_empty() || component ==
```

```
"." || component == ".." {  
    bail!("'{component}' is not a valid  
name");  
}  
if !component.chars().all(|c| {  
    c.is_ascii_alphanumeric() || matches!  
(c, '-' | '_' | '.')  
}) {  
    bail!("'{component}' may only contain  
safe characters");  
}  
Ok(())  
}
```

Security tools don't usually fail because the main idea is wrong. They fail because some edge path quietly bypasses the idea.

The takeaway

The most useful design sentence in this project was:

Access control is the crypto.

Once that was written down, many decisions became simpler. No permission flags. No second encryption path. No plaintext descriptions. No "temporary" bypass around signed

members.

The architecture is not more flexible because of that constraint. It is less flexible in exactly the ways that make it easier to trust.

For me, the product idea is not "another secrets manager."

It is: **stop renting access to small-team secrets when Git + SSH keys are already enough.** Keep ciphertext in your repo, keep private keys on your machines, add/remove teammates by changing recipients, and let both humans and agents use the same workflow.

Repo: <https://github.com/mistral/sshare>

#rust #security #cli #cryptography #devtools